

Thread Synchronization Problems

High Performance Computing
Master in Applied Artificial Intelligence
Nuno Lopes

Thread Execution Ordering

- ▶ The Mutual Exclusion mechanism makes it possible to guarantee that at no time does more than one thread execute "simultaneously" code that manipulates the same resource.
- ▶ Some problems are based on the ordering of thread execution.
 - ▶ Threads have an implicit execution order between them, even though they are concurrent.
- ▶ Typical cases of synchronization:
 - ▶ Producer - Consumer
 - ▶ Reader - Writer
 - ▶ Barrier

Producer-Consumer

- ▶ Problem consists of two types of threads: Producer and Consumer.
- ▶ Producer - produces resources (to be shared)
 - ▶ Production capacity depends on the availability of storing produced resources
 - ▶ Stores the produced resource in a shared buffer
- ▶ Consumer - consumes produced resources
 - ▶ Condition: only consumed resources already produced can be consumed
 - ▶ Removes consumed resources from the shared buffer

Reader-Writer

- ▶ In this problem there are threads of two types:
 - ▶ Readers: threads that read data
 - ▶ Writers: threads that change the data
- ▶ The restrictions of this problem are the following:
 - ▶ several readers can access the data concurrently, without risk of creating inconsistencies;
 - ▶ when a writer wants to alter the data there can be no other simultaneous access, reading or writing.

Barrier

- ▶ Problem of synchronization between threads.
- ▶ Threads invoke `barrier()` function, which blocks the execution of the thread that calls it, until all threads are in this position.
- ▶ When all threads execute this function, the function terminates (and unblocks) for all threads.
- ▶ It acts as a synchronization point in the execution of each thread.

Condition Variables

Barrier Example using Active Waiting

```
/* Shared and initialized by the main thread */
int counter; /* Initialize to 0 */
int thread_count;
pthread_mutex_t barrier_mutex;
. . .

void* Thread_work(. . .) {
    . . .
    /* Barrier */
    pthread_mutex_lock(&barrier_mutex);
    counter++;
    pthread_mutex_unlock(&barrier_mutex);
    while (counter < thread_count);
    . . .
}
```

Motivation for Condition Variables

- ▶ Sometimes the need arises in threads to check that a condition (global to all threads) is fulfilled.
- ▶ However:
 - ▶ it is necessary to have access to a common global state;
 - ▶ but it is also necessary to wait for this state to be changed by another thread.

Motivation for Condition Variables

- ▶ Now, to make an access it is necessary to use *Mutex*, but as long as the thread has the *Mutex* it does not allow the others to change the (same) global state.
- ▶ On the other hand, for the thread to wait for the state to change, in order to meet the condition, it is necessary to check if it has changed:
 - ▶ Active waiting!
- ▶ How to do this synchronisation of a global state without active wait, nor block the threads ?

Condition Variable

- ▶ A condition variable is a structure that allows threads to suspend execution until a certain event (or condition) occurs.
- ▶ When the event occurs, a signalling mechanism will "wake up" the locked threads to continue execution.
- ▶ A condition variable must be associated with a mutual exclusion (mutex) mechanism.

Condition Variables API

- ▶ Passive waiting:
 `pthread_cond_wait (cv, mt)`
- ▶ Signaling:
 `pthread_cond_signal (cv)`
 `pthread_cond_broadcast (cv)`
- ▶ Creation and destruction:
 `pthread_cond_init ()`
 `pthread_cond_destroy ()`
- ▶ Auxiliary Structure:
 `pthread_cond_t`

Condition Variables API

- ▶ Function for waiting for the event to occur:

```
int pthread_cond_wait(  
    pthread_cond_t* cond_var_p    /* in/out */,  
    pthread_mutex_t* mutex_p      /* in/out */ );
```

- ▶ This function has a combined and atomic function of the following functions:
 - ▶ pthread_mutex_unlock(mutex);
 - ▶ wait for signal on condition variable;
 - ▶ pthread_mutex_lock(mutex);

Condition Variables API

- ▶ Function to signal the occurrence of the event:

For (any) locked thread:

```
int pthread_cond_signal(  
    pthread_cond_t* cond_var_p /* in/out */);
```

For all locked threads:

```
int pthread_cond_broadcast (  
    pthread_cond_t* cond_var_p /* in/out */);
```

Condition Variables API

- ▶ Type of auxiliary structure :

pthread_cond_t

- ▶ Structure initialisation and termination functions

```
int pthread_cond_init(  
    pthread_cond_t* cond_p          /* out */,  
    const pthread_condattr_t* cond_attr_p /* in */);  
  
int pthread_cond_destroy(  
    pthread_cond_t* cond_p          /* in/out */);
```

Condition Variables Usage Pattern

- ▶ Problem of synchronization between threads, without using active wait.
- ▶ Threads invoke **wait()** function, which only returns when (the) event is signalled.
- ▶ All threads update global state (with mutex), and block.
- ▶ The last thread, which detects the event or condition is correct, signals to all: **broadcast()**

Barrier Example using Condition Variables (from Pacheco)

```
/* Shared */
int counter = 0;
pthread_mutex_t mutex;
pthread_cond_t cond_var;
. . .
void* Thread_work(. . .) {
    . . .
    /* Barrier */
    pthread_mutex_lock(&mutex);
    counter++;
    if (counter == thread_count) {
        counter = 0;
        pthread_cond_broadcast(&cond_var);
    } else {
        while (pthread_cond_wait(&cond_var, &mutex) != 0);
    }
    pthread_mutex_unlock(&mutex);
    . . .
}
```


Barrier Example using Condition Variables (from Pacheco)

Barrier Pseudocode:

```
Mutex_lock(mutex);  
counter++;  
  
if(counter == thread_count):  
    counter = 0;  
    Cond_broadcast( cond_var );  
else: //(counter < thread_count)  
    while (...);  
    cond_wait( cond_var, mutex )  
  
Mutex_unlock(mutex);
```

```
/* Shared */  
int counter = 0;  
pthread_mutex_t mutex;  
pthread_cond_t cond_var;  
...  
void* Thread_work(. . .) {  
    ...  
    /* Barrier */  
    pthread_mutex_lock(&mutex);  
    counter++;  
    if (counter == thread_count) {  
        counter = 0;  
        pthread_cond_broadcast(&cond_var);  
    } else {  
        while (pthread_cond_wait(&cond_var, &mutex)  
        )  
        pthread_mutex_unlock(&mutex);  
        ...  
    }  
}
```

Producer-Consumer Example

Producer-Consumer UnBounded Buffer Example

- ▶ An example of producer-consumer synchronization problem is the unbounded buffer:
 - ▶ Thread Producer generates items for a buffer (list) with limited size;
 - ▶ Thread Consumer extracts items from that same list.
- ▶ Cases:
 - ▶ Consumer can only execute when there is at least 1 item in the list;
 - ▶ Producer can only execute if the list has space available for 1 more item (assuming limited size).

Producer-Consumer UnBounded Buffer Example (pseudo-code)

```
mutex_t mutex;    // mutex_init()
cond_t notEmptyforConsumer; // cond_init()
cond_t notFullforProducer;    // cond_init()
```

```
void* buffer[];
int firstptr, lastptr, count; // = 0
int buf_size; // buffer size
```

```
void* produce(void* item);
```

```
void* consume();
```

Producer-Consumer UnBounded Buffer Example (pseudo-code)

```
void* produce(void* item) {  
    mutex_lock( &mutex);  
    while (count == buffer_size)  
        pthread_cond_wait( &mutex, &notFullforProducer );  
    buffer[lastptr] = item;  
    lastptr = lastptr + 1 % buffer_size;  
    count += 1;  
    pthread_cond_signal( &notEmptyforConsumer );  
    mutex_unlock( &mutex);  
} // end produce
```

Producer-Consumer UnBounded Buffer Example (pseudo-code)

```
void* consume() {  
    mutex_lock( &mutex);  
    while (count == 0)  
        pthread_cond_wait( &mutex, &notEmptyforConsumer);  
    item = buffer[firstptr];  
    firstptr = firstptr + 1 % buffer_size;  
    count -= 1;  
    pthread_cond_signal( &notFullforProducer );  
    mutex_unlock( &mutex);  
    return item;  
} // end consume
```